Data Visualization and Introduction to Streamlit
IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

Adapted from the Streamlit documentation, https://docs.streamlit.io, and 30 Days of Streamlit, https://30days.streamlit.app.

1. Why Streamlit
2. Running an App
3. The Execution Model
4. Putting Things on the Page

Why Streamlit

The Gap in What We Have Built

- We can put data into MySQL and query it. We can put data into Redis and read it back quickly. We can package all of it into containers that run anywhere.
- What we cannot do yet is hand any of it to somebody else. Every result so far has appeared in a Jupyter notebook, which requires the reader to run Python, read code, and know which cell to execute.
- A warehouse manager is not going to open a notebook. They want a page with a dropdown and a chart.
- Building that page has traditionally meant HTML, CSS, JavaScript, and a web framework, which is a second skill set on top of the one this course teaches.

What Streamlit Is

- Streamlit is an open-source Python library that turns a plain Python script into a web application.
- There is no HTML, no CSS, no JavaScript, and no callback plumbing to write. You write a script top to bottom, and the widgets and charts you call appear on a page.
- It was built for exactly our situation: people who can write Python and work with data, but who are not web developers.
- It is one of the four textbooks for this course. The documentation at `https://docs.streamlit.io` is the reference, and 30 Days of Streamlit is a guided tour of one command per day.

You Have Already Run One

In Lecture 4, the port forwarding example ran this file inside a container. At the time the point was the port mapping, and the contents of the script were beside the point.

```
import streamlit as st

st.title("Hi from streamlit inside docker")

def submit_name():
    st.session_state.submitted = True

st.text_input("Please enter your name:",
              key="name", on_change=submit_name)

if st.session_state.get("submitted", False):
    st.write(f"My name is: {st.session_state.name}")
```

- Fourteen lines, no HTML, and it is a working web page with a text box that reacts to what you type.
- By the end of the next lecture every line of this will make sense, including why `session_state` has to be there at all.

Source: `docker_tutorial/networking/app/main.py`

Running an App

Running an App

A Streamlit app is an ordinary `.py` file. What makes it an app is how you start it.

```
pip install streamlit
streamlit run app.py
```

- Note that you do not run it with `python app.py`. That executes the script once and exits, printing nothing useful. The `streamlit run` command starts a small web server that executes the script and serves the result.
- The app appears at `localhost:8501`. Port 8501 is to Streamlit what 3306 is to MySQL and 6379 is to Redis.
- Saving the file causes the browser to offer a rerun, so the edit and refresh loop is immediate.

Running an App in a Container

Since the app is a Python script with a dependency, it containerizes exactly the way everything else in this course has.

```
FROM python:3.12
WORKDIR /usr/src/app
COPY ./requirements.txt /usr/src/app/requirements.txt
RUN pip install -r requirements.txt
COPY ./ /usr/src/app
CMD streamlit run app/main.py --server.port 8501
```

- The `requirements.txt` for this image contains one line, `streamlit`.
- The container listens on 8501, so reaching it from the host needs a published port, `-p 8501:8501`, exactly as in Lecture 4. Without that mapping the app runs perfectly and nobody can see it.

Source: `docker_tutorial/networking/Dockerfile`

Bringing Up the Course Examples

The examples for this lecture and the next two live in `docker_tutorial/streamlit-dev/`, one folder each.

```
cd docker_tutorial/streamlit-dev/lecture16
docker compose up -d
```

Then open `http://localhost:8503`.

Folder	What it shows	Open
`lecture16/`	Text, tables, metrics, charts	`localhost:8503`
`lecture17/`	Widgets, layout, session state	`localhost:8504`
`lecture18/`	Dashboard over MySQL and Redis	`localhost:8505`

- Different published ports, so all three can run at once.
- `docker compose down` in the folder you started stops it again.

One Image, Several Examples

Nothing above is built. Each folder pulls the same prebuilt image and only changes the command it runs.

```
services:
  streamlit:
    image: ghcr.io/disys-lab/streamlit-base:latest
    ports:
      - "8503:8501"
    volumes:
      - ./app:/usr/src/app/app
    command: >
      streamlit run app/basics.py
        --server.port 8501
        --server.address 0.0.0.0
```

- The image is Python with `streamlit`, `pandas`, `pymysql` and `redis` already installed, so there is no wait for `pip` on first run.
- app/ is mounted from your machine, so editing a file and saving it updates the page immediately. You only rebuild if you add a package.
- Lecture 18 adds `mysql` and `redis` services to the same file, and needs seeding once with `docker compose exec streamlit python app/seed.py`.

The Execution Model

The Rerun Model

- This is the one idea in Streamlit that is genuinely different from ordinary Python, and almost every confusion with the library traces back to it.
- **When anything needs to change on screen, Streamlit reruns your entire script from top to bottom.**
- That happens when you edit and save the source, and it happens every time a user touches a widget. Move a slider one notch and the whole script runs again.
- There is no event handler to register and no part of the page you update by hand. You write the script as though it described the page from scratch, because every time it runs, it does.

Source: Streamlit documentation, Basic concepts of Streamlit

Why This Is Surprising

- It means an ordinary Python variable cannot remember anything. A counter you increment is back to its starting value on the next interaction, because the line that created it just ran again.
- It means expensive work gets repeated. If line three loads a CSV or opens a database connection, that happens again on every single click.
- Both problems have answers, and they are the two features that distinguish a working Streamlit app from a toy. Caching solves the second one and is covered below. Session state solves the first and is the subject of the next lecture.
- The payoff is that the model is extremely simple to reason about. There is no hidden state and no ordering of events to keep track of, because there is only ever one path through the script.

Widgets Are Variables

The rerun model produces an unusual and rather elegant consequence.

```
import streamlit as st

x = st.slider("Pick a number", 0, 100)
st.write(x, "squared is", x * x)
```

- `st.slider` draws the slider *and* returns its current value, so `x` is just an integer.
- Drag the slider and the script reruns with the new value of `x`, so the line below it recomputes and the page updates. No callback was written.
- Every input widget in Streamlit works this way: it returns what the user has chosen, and the rest of the script is ordinary Python.

Caching: Two Decorators

If the whole script reruns on every click, loading a file or opening a connection every time is unacceptable. Streamlit's answer is a decorator.

```
@st.cache_data
def load_orders():
    return pd.read_csv("data/orders/CustomerOrder.csv")

@st.cache_resource
def get_connection():
    return pymysql.connect(host="mysql", ...)
```

- `@st.cache_data` is for **data**: dataframes, lists, query results. It hands back a fresh copy each call, so one user cannot corrupt another's.
- `@st.cache_resource` is for **things you open once**: database connections, loaded models. It returns the same object to everyone, which is what you want for a connection and disastrous for a dataframe.
- Both take `ttl` and `max_entries`. `@st.cache_data(ttl=300)` is the Redis caching pattern from Lecture 12, with the same staleness trade-off.

Source: Streamlit documentation, Caching

Putting Things on the Page

Magic, and st.write

```
df # magic: a bare variable displays itself
st.write(df) # the same thing, said explicitly
st.write("Hello", 42, df, fig)
```

- Any variable alone on its own line is written to the page. Streamlit calls this magic, and it is useful for quick work.
- `st.write` is described in the documentation as Streamlit's Swiss Army knife. It inspects what you give it and chooses a sensible rendering, whether that is text, a dataframe, a chart or a figure.
- Prefer `st.write` in anything you intend to keep. It is explicit, it accepts several arguments at once, and a reader does not have to know the magic rule.

Text and Structure

```
st.title("Warehouse Dashboard")
st.header("Orders")
st.subheader("By carrier")
st.markdown("Shipments **late** by more than one day.")
st.caption("Source: WMS, refreshed every 5 minutes.")
st.code("SELECT * FROM customerorder;", language="sql")
st.metric("Open orders", 128, delta=-7)
```

- These are the headings and running text of the page, and they are the cheapest way to make a dashboard readable.
- `st.metric` is worth knowing early. It renders a single number large, with an optional change indicator, which is what most dashboards actually need at the top.

Showing a Table

```
st.dataframe(df) # interactive: sort, resize, scroll
st.table(df) # static: the whole thing, no controls
st.dataframe(df, use_container_width=True)
```

- `st.dataframe` gives the user a table they can sort and scroll, and it handles a large result sensibly. This is almost always the one you want.
- `st.table` writes the entire table out statically. It suits a small summary of a few rows and is a poor choice for a query result of unknown size.
- Both accept a pandas DataFrame, a NumPy array, or a plain list, so the output of a `cursor.fetchall()` can go straight to the page.

The Built-in Charts

```
st.line_chart(df, x="Order_Date", y="Order_Count")
st.bar_chart(df, x="Carrier", y="Shipments")
st.area_chart(df)
st.scatter_chart(df, x="Item_Price", y="Item_AnnualSales")
st.map(df) # needs latitude and longitude columns
```

- These five are the native charts. They take a dataframe, they need almost no configuration, and they are interactive by default.
- They are deliberately limited. When you need control over the result, Streamlit wraps the usual libraries instead: `st.pyplot` for Matplotlib, and `st.altair_chart`, `st.plotly_chart`, `st.pydeck_chart` for the others.
- The rule of thumb is to start with the native chart and move to a wrapper only when you hit something it cannot express.

Source: Streamlit documentation, API reference, Chart elements

Where This Is Going

- You can now write a page that loads data, shows it as a table, and draws a chart, and you can run that page inside a container.
- What it does not yet do is let anyone change what they are looking at. That needs input widgets, and a layout to put them in.
- It also cannot remember anything between clicks, which the rerun model guarantees, and which session state exists to fix.
- Both are the subject of the next lecture, after which the Lecture 4 example will read as ordinary code rather than as a trick.